

Practical 1

AIM: Design Login activity and implement control events: Use EditText, Checkbox and Buttons. And implement above events using following layouts: 1. Linear Layout 2. Relative Layout 3. Table Layout

Date:

Relevant CO: CO2

Theory :

➤ **LinearLayout:**

- **Orientation:** LinearLayout is a simple and straightforward layout that arranges its child views either horizontally or vertically, depending on the specified orientation (horizontal or vertical).
- **Child View Alignment:** Child views are positioned in a linear manner one after the other, and you can control their alignment within the layout, such as left, center, right (for horizontal orientation) or top, center, bottom (for vertical orientation).
- **Weighted Distribution:** You can assign weights to child views, which allows you to control how much space each view occupies relative to others. This is useful for creating flexible layouts where views can expand or shrink based on screen size or content.
- **Nested Layouts:** LinearLayout can be nested within other layouts, such as inside a ScrollView or as a child of other ViewGroup layouts, to create complex UI designs.

➤ **RelativeLayout:**

- **Positioning Based on Relationships:** RelativeLayout allows you to position child views relative to each other or relative to the parent layout. You define the position of a view in relation to another view using attributes like `android:layout_above`, `android:layout_below`, `android:layout_toLeftOf`, `android:layout_toRightOf`, etc.
- **Dynamic Layouts:** RelativeLayout is great for creating dynamic and responsive layouts that adapt to different screen sizes and orientations. It's particularly useful when you don't know the exact size or position of views in advance.
- **Complex Hierarchies:** You can create complex view hierarchies with RelativeLayout by specifying rules for each view, allowing for intricate and customized UI designs.

➤ **TableLayout:**

- **Grid-Based Layout:** TableLayout is used to create a grid-like layout where child views are organized in rows and columns. It's especially useful when you need to display data or elements in a tabular format.
- **Rows and Columns:** You define the structure of the table by adding TableRow elements as child views of TableLayout. Each TableRow represents a row in the table, and you can add child views to these TableRow elements, which act as cells within the table.
- **Uniform Sizing:** TableLayout distributes space evenly among columns and rows, ensuring that cells have uniform sizes. You can customize the width of columns or the height of rows if needed.
- **Data Display:** TableLayout is commonly used for displaying data sets or grids of content, such as contact lists, calendars, or data input forms.

CODE:-

XML File

➤ **LinearLayout:**

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    android:layout_marginHorizontal="10dp"
```

```
    android:gravity="center_vertical"
```

```
    tools:context=".MainActivity">
```

```
<ImageView
```

```
    android:layout_width="120dp"
```

```
    android:layout_height="120dp"
```

```
    android:layout_gravity="center"
```

```
    android:src="@drawable/images"
```

```
android:contentDescription="College Logo"  
android:scaleType="fitCenter" />
```

```
<TextView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Shree Swami Atmanand Saraswati Institute of Technology"  
    android:gravity="center"  
    android:textColor="@color/design_default_color_primary_variant"  
    android:textStyle="bold"  
    android:textSize="20sp"  
    android:layout_marginTop="10dp" />
```

```
<EditText  
    android:id="@+id/userNameEditText"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Enter User Name"  
    android:layout_marginTop="10dp" />
```

```
<EditText  
    android:id="@+id/userPasswordEditText"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Enter Your Password"  
    android:layout_marginTop="10dp"  
    android:inputType="textPassword" />
```

```
<CheckBox  
    android:id="@+id/verificationCheckBox"
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="I am not a robot"
    android:layout_marginTop="10dp" />
```

```
<Button
    android:id="@+id/loginBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:text="LOGIN" />
```

```
</LinearLayout>
```

➤ **RelativeLayout:**

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginHorizontal="20dp"
    tools:context=".MainActivity">
```

```
<ImageView
    android:id="@+id/collegeImage"
    android:layout_width="120dp"
    android:layout_height="120dp"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:src="@drawable/images"
    android:contentDescription="College Logo"
```

```
        android:scaleType="fitCenter" />
```

```
<TextView
```

```
    android:id="@+id/collegeName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Shree Swami Atmanand Saraswati Institute of Technology"
    android:textColor="@color/design_default_color_primary_variant"
    android:textStyle="bold"
    android:textSize="20sp"
    android:gravity="center"
    android:layout_below="@id/collegeImage"
    android:layout_marginTop="10dp" />
```

```
<EditText
```

```
    android:id="@+id/userNameEditText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter User Name"
    android:layout_below="@id/collegeName"
    android:layout_marginTop="10dp" />
```

```
<EditText
```

```
    android:id="@+id/userPasswordEditText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Your Password"
    android:inputType="textPassword"
    android:layout_below="@id/userNameEditText"
    android:layout_marginTop="10dp" />
```

```
<CheckBox
```

```
    android:id="@+id/verificationCheckBox"
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
    android:text="I am not a robot"
    android:layout_below="@id/userPasswordEditText"
    android:layout_marginTop="10dp" />
```

```
<Button
```

```
    android:id="@+id/loginBtn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="LOGIN"
    android:layout_below="@id/verificationCheckBox"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="10dp" />
```

```
</RelativeLayout>
```

➤ **Table Layout:**

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<TableLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="20dp"
    android:stretchColumns="1"
    tools:context=".MainActivity">
```

```
<TableRow>
```

```
<ImageView
```

```
    android:id="@+id/collegeImage"
    android:layout_width="120dp"
    android:layout_height="120dp"
```

```
        android:layout_span="2"
        android:layout_gravity="center"
        android:src="@drawable/images"
        android:contentDescription="College Logo"
        android:scaleType="fitCenter" />
```

```
</TableRow>
```

```
<TableRow>
```

```
    <TextView
        android:id="@+id/collegeName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_span="2"
        android:text="Shree Swami Atmanand Saraswati Institute of Technology"
        android:textColor="@color/black"
        android:textStyle="bold"
        android:gravity="center"
        android:textSize="20sp" />
```

```
</TableRow>
```

```
<TableRow>
```

```
    <EditText
        android:id="@+id/userNameEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter User Name"
        android:layout_span="2" />
```

```
</TableRow>
```

```
<TableRow>
```

```
<EditText
```

```
    android:id="@+id/userPasswordEditText"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:hint="Enter Your Password"
```

```
    android:inputType="textPassword"
```

```
    android:layout_span="2" />
```

```
</TableRow>
```

```
<TableRow>
```

```
<CheckBox
```

```
    android:id="@+id/verificationCheckBox"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="I am not a robot"
```

```
    android:layout_span="2" />
```

```
</TableRow>
```

```
<TableRow>
```

```
<Button
```

```
    android:id="@+id/loginBtn"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_span="2"
```

```
    android:layout_gravity="center"
```

```
    android:text="LOGIN" />
```

```
</TableRow>
```

```
</TableLayout>
```

JAVA:

```
package com.example.practical1;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import com.example.practical1.R;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_relative_layout);
        // setContentView(R.layout.activity_main);
        // setContentView(R.layout.activity_table_layout);
        // different layout uses

        EditText userName, userPassword;
        Button loginBtn;

        userName = findViewById(R.id.userNameEditText);
        userPassword = findViewById(R.id.userPasswordEditText);
        loginBtn = findViewById(R.id.loginBtn);

        loginBtn.setOnClickListener(new View.OnClickListener() {

            @Override
```

```

public void onClick(View view) {
    if (userName.getText().toString().equals("User") &&
        userPassword.getText().toString().equals("12345")) {
        Toast.makeText(MainActivity.this, "Login Successful",
            Toast.LENGTH_SHORT).show();
    } else {

        Toast.makeText(MainActivity.this, "Incorrect username or password",
            Toast.LENGTH_SHORT).show();
    }
}
});
}
}

```

OutPut:

 Shree Swami Atmanand Saraswati Institute of Technology Enter User Name Enter Your Password ☐ I am not a robot LOGIN	 Shree Swami Atmanand Saraswati Institute of Technology Enter User Name Enter Your Password ☐ I am not a robot LOGIN	 Shree Swami Atmanand Saraswati Institute of Technology Enter User Name Enter Your Password ☐ I am not a robot LOGIN
---	---	---

LinearLayout

RelationLayout

TableLayout

Practical 2

AIM: Create Activities & implement following

1. Implicit intent 2. Explicit Intent 3. Start Activity for Result

Date:

Relevant CO: CO1, CO2

Theory :

➤ **Explicit Intent:**

- **Target Component Specified:** An explicit intent is used when you know exactly which component (usually an activity) you want to start. You provide the fully qualified class name of the target component.
- **Specific Component:** You explicitly define the component to be launched within your app, making it suitable for navigating within your own app's various screens or functionalities.
- **Common Use Cases:** Explicit intents are commonly used for navigating within an app, launching specific activities, and passing data between them.
- **Strongly Typed:** Explicit intents are strongly typed, meaning you have compile-time checks, and Android Studio can validate whether the target component exists and whether the data types match.

➤ **Implicit Intent:**

- **Target Component Not Specified:** An implicit intent is used when you want to perform an action, but you do not specify the exact component to handle it. Instead, you define the action, data, and type, and Android system resolves the appropriate component.
- **Dynamic Component Resolution:** Implicit intents are suitable for scenarios where multiple components within the device or other installed apps can respond to the specified action and data type.
- **Common Use Cases:** Implicit intents are used for actions that can be performed by various apps or components, such as opening a web link, sharing content, or sending an email.

- **Runtime Resolution:** Android's system resolves which component should handle the implicit intent at runtime, based on the action, data, and type specified. This allows for greater flexibility.
- **Fallback Mechanism:** If no suitable component is found to handle the implicit intent, your app won't crash; it can gracefully handle the absence of a capable component.

Code:-**Activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:layout_marginHorizontal="10dp"
    android:layout_marginTop="20dp">

    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter URL" />

    <Button
        android:id="@+id/explicitButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="EXPLICIT SEARCH" />

    <Button
        android:id="@+id/implicitButton"
```

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="IMPLICIT BUTTON" />
```

```
<Button
    android:id="@+id/resultButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="ACTIVITY FOR RESULT" />
```

```
</LinearLayout>
```

Activity_explicit_intent.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_marginHorizontal="20dp">
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="This is Explicit Activity"
    android:textSize="20sp" />
```

```
<Button
    android:id="@+id/backButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="BACK"
```

```
        android:layout_marginTop="10dp" />
```

```
</LinearLayout>
```

Activity_for_result.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    android:gravity="center">
```

```
    <TextView
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="Activity For Result"
```

```
        android:textSize="20sp" />
```

```
    <Button
```

```
        android:id="@+id/sendButton"
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="SEND RESULT" />
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.practical2;
```

```
import androidx.activity.result.ActivityResultLauncher;
```

```
import androidx.activity.result.contract.ActivityResultContracts;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;

import com.example.practical2.ActivityForResult;
import com.example.practical2.R;

public class MainActivity extends AppCompatActivity {

    EditText editText;
    Button explicitButton, implicitButton, resultButton;

    // Activity For Result
    ActivityResultLauncher<Intent> resultLauncher =
        registerForActivityResult(
            new ActivityResultContracts.StartActivityForResult(), result -> {

                if (result.getResultCode() == RESULT_OK && result.getData() !=
null) {

                    String value = result.getData().getStringExtra("result");

                    editText.setText(value);
                }
            }
        );

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

```
editText = findViewById(R.id.editText);
explicitButton = findViewById(R.id.explicitButton);
implicitButton = findViewById(R.id.implicitButton);
resultButton = findViewById(R.id.resultButton);

// Explicit Intent
explicitButton.setOnClickListener(view -> {

    Intent intent = new Intent(MainActivity.this, ExplicitIntentActivity1.class);

    startActivity(intent);
});

// Implicit Intent
implicitButton.setOnClickListener(view -> {

    String url = editText.getText().toString();

    if (!url.startsWith("http://") && !url.startsWith("https://")) {
        url = "https://" + url;
    }

    Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(url));

    startActivity(intent);
});

// Start Activity For Result
resultButton.setOnClickListener(view -> {

    Intent intent = new Intent(MainActivity.this, ActivityForResult.class
    );

    resultLauncher.launch(intent);
});
```

```
}  
}
```

ExplicitIntentActivity1.java

```
package com.example.practical2;  
  
import androidx.appcompat.app.AppCompatActivity;  
  
import android.os.Bundle;  
import android.widget.Button;  
  
public class ExplicitIntentActivity1 extends AppCompatActivity {  
  
    Button backButton;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        setContentView(R.layout.activity_explicit_intent1);  
        backButton = findViewById(R.id.backButton);  
        backButton.setOnClickListener(view -> {  
            finish();  
        });  
    }  
}
```

ActivityResult.java

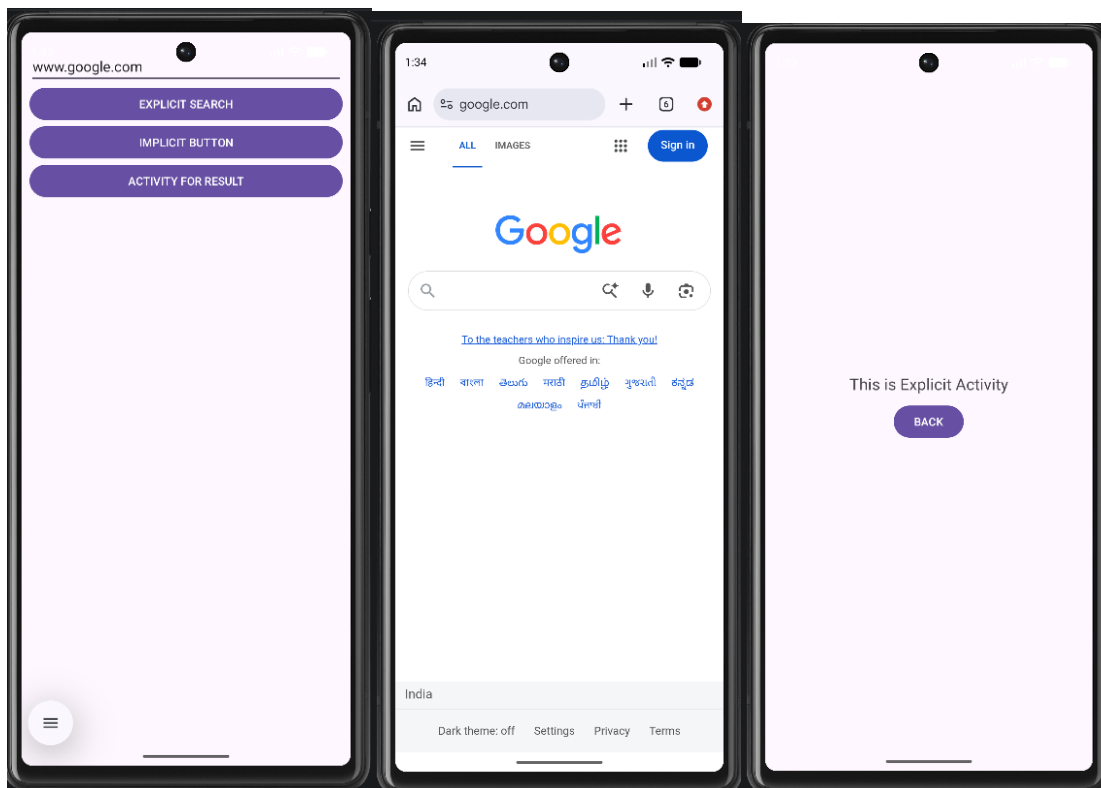
```
package com.example.practical2;  
  
import androidx.appcompat.app.AppCompatActivity;  
  
import android.content.Intent;  
import android.os.Bundle;  
import android.widget.Button;
```

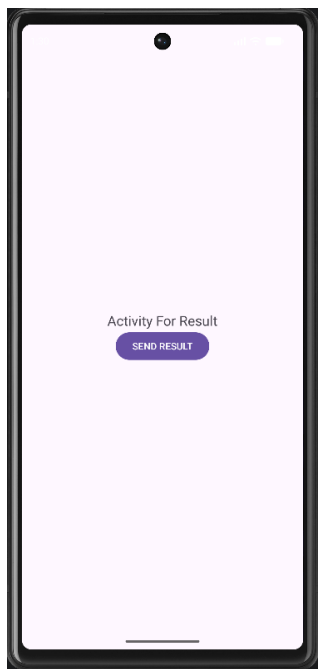
```

public class ActivityForResult extends AppCompatActivity {
    Button sendButton;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_for_result);

        sendButton = findViewById(R.id.sendButton);
        sendButton.setOnClickListener(view -> {
            Intent intent = new Intent();
            intent.putExtra("result", "Result Received");
            setResult(RESULT_OK, intent);
            finish();
        });
    }
}

```





Practical 3

AIM: Use an Options Menu.

Date:

Relevant CO: CO2

Theory :

The Option Menu in Android Studio, also known as the "Overflow Menu" or "Action Bar Menu," is a common user interface element used to provide access to various actions, settings, and additional functionality within an Android app. Here's an explanation of the Option Menu, point by point:

➤ **Purpose:**

- The Option Menu provides a way to access app-specific actions and settings, typically displayed as a three-dot icon (⋮) in the top-right corner of the app's action bar (also known as the app bar or toolbar).
- Access: Users can access the Option Menu by tapping the three-dot icon or by pressing the device's menu button (if available) when it's supported. In modern Android app design, the menu button on devices has largely been replaced by onscreen navigation.
- Items: The Option Menu consists of a list of items, each representing a specific action, command, or setting. These items can be text-based or include icons.
- Grouping: Menu items are typically organized into groups for better categorization and to make it easier for users to find the desired action. Common groups include "Settings," "Share," "Help," and "About."
- Inflating the Menu: To create and define the Option Menu, you override the `onCreateOptionsMenu` method in your activity or fragment. You inflate a menu XML resource that defines the menu items and their properties.
- Handling Selection: You can specify callback methods to handle user selection of menu items. The `onOptionsItemSelected` method is used to define what should happen when a menu item is selected.
- Icons and Icons with Text: Menu items can include icons to provide visual cues for the actions. Android allows you to use both icons alone or icons accompanied by text labels.

- **Dynamic Changes:** You can dynamically change the contents of the Option Menu based on the app's state. This is useful for enabling or disabling specific actions or updating menu item labels.
- **Overflow Menu:** When the action bar lacks space to display all menu items, some items may be moved to the Overflow Menu, which is accessible by tapping the three dot icon. This allows for a cleaner and more accessible user interface. **Compatibility:** The Option Menu is designed to be compatible with various Android versions. It adapts its appearance and behavior depending on the version of Android the app is running on.
- **Customization:** You can customize the appearance of the Option Menu, including changing the background color, text color, and more to match your app's design and branding.
- **Best Practices:** When implementing the Option Menu, consider the Android design guidelines to ensure a consistent and user-friendly experience. Keep the menu items concise, use appropriate icons, and organize actions logically.
- **Alternative Menu Types:** In addition to the Option Menu, Android supports other menu types like the Popup Menu and the Bottom App Bar for different UI designs and use cases.

Code:**Activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical"

    android:paddingTop="30dp">
```

```
<androidx.appcompat.widget.Toolbar
    android:id="@+id/options_menu_tb"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:minHeight="?attr/actionBarSize"
    android:background="?attr/colorPrimary"
    app:title="Options Menu"
    app:titleTextColor="#FFFFFF" />
</LinearLayout>
```

Option_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/item1"
        android:title="Setting" />
    <item
        android:id="@+id/item2"
        android:title="About Us" />
</menu>
```

JAVA

```
package com.example.practical3;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
```

```
import androidx.appcompat.widget.Toolbar;
```

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        Toolbar toolbar = findViewById(R.id.options_menu_tb);
```

```
        setSupportActionBar(toolbar);
```

```
    }
```

```
    @Override
```

```
    public boolean onCreateOptionsMenu(Menu menu) {
```

```
        getMenuInflater().inflate(R.menu.options_menu, menu);
```

```
        return true;
```

```
    }
```

```
    @Override
```

```
    public boolean onOptionsItemSelected(@NonNull MenuItem item) {
```

```
        if (item.getItemId() == R.id.item1) {
```

```
            Toast.makeText(this, "Setting Selected", Toast.LENGTH_SHORT).show();
```

```
            return true;
```

```
        }
```

```
        if (item.getItemId() == R.id.item2) {
```

```
            Toast.makeText(this, "About Us Selected", Toast.LENGTH_SHORT).show();
```

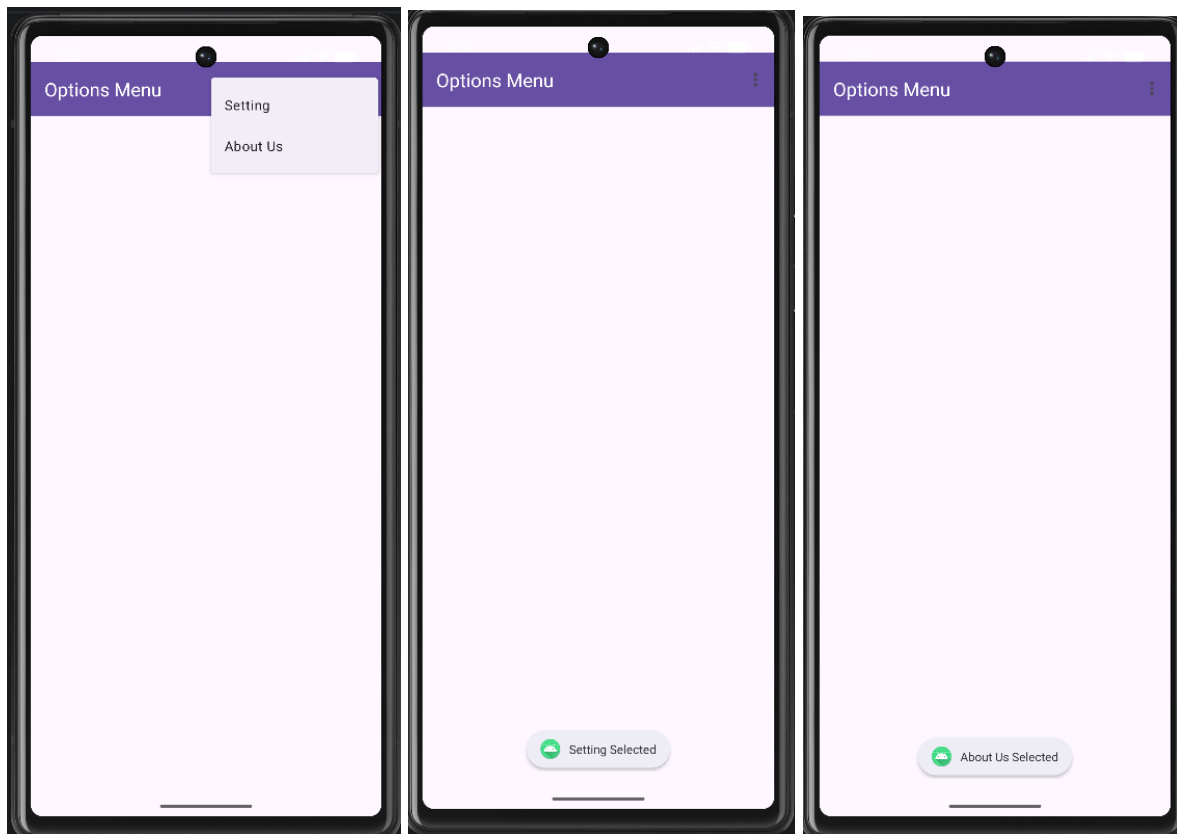
```
            return true;
```

```
        }
```

```
        return super.onOptionsItemSelected(item);
```

```
    }
```

```
}
```



Practical 4

AIM : Create a Recycler View and list the details of student using following fields: 1. Name 2. Address 3. Photo (Image) 4. Delete (Button Operation)

Date:

Relevant CO: CO1, CO2

Theory :

RecyclerView is a ViewGroup added to the android studio as a successor of the GridView and ListView. It is an improvement on both and can be found in the latest v-7 support packages. It has been created to make possible construction of any lists with XML layouts as an item which can be customized vastly while improving on the efficiency of ListViews and GridViews. This improvement is achieved by recycling the views which are out of the visibility of the user.

For example, if a user scrolled down to a position where items 4 and 5 are visible; items 1, 2, and 3 would be cleared from the memory to reduce memory consumption.

To implement a basic RecyclerView three sub-parts are needed to be constructed which offer the users the degree of control they require in making varying designs of their choice.

The Card Layout: The card layout is an XML layout which will be treated as an item for the list created by the RecyclerView.

The ViewHolder: The ViewHolder is a java class that stores the reference to the card layout views that must be dynamically modified during the execution of the program by a list of data obtained either by online databases or added in some other way.

The Data Class: The Data class is a custom java class that acts as a structure for holding the information for every item of the RecyclerView.

Steps for implementing your RecyclerView

If you are going to use RecyclerView, there are a few things you need to do. They are explained in detail in the following sections.

1. Decide how the list or grid looks. Ordinarily, you can use one of the RecyclerView library's standard layout managers.
2. Design how each element in the list looks and behaves. Based on this design, extend the ViewHolder class. Your version of ViewHolder provides all the functionality for your list items. Your view holder is a wrapper around a View, and that view is managed by RecyclerView.
3. Define the Adapter that associates your data with the ViewHolder views. There are also advanced customization options that let you tailor your RecyclerView to your exact needs.

Plan your layout

The items in your RecyclerView are arranged by a LayoutManager class. The RecyclerView library provides three layout managers, which handle the most common layout situations:

- LinearLayoutManager arranges the items in a one-dimensional list.
- GridLayoutManager arranges the items in a two-dimensional grid:
 - If the grid is arranged vertically, GridLayoutManager tries to make all the elements in each row have the same width and height, but different rows can have different heights.
 - If the grid is arranged horizontally, GridLayoutManager tries to make all the elements in each column have the same width and height, but different columns can have different widths.
- StaggeredGridLayoutManager is similar to GridLayoutManager, but it does not require that items in a row have the same height (for vertical grids) or items in the same column have the same width (for horizontal grids). The result is that the items in a row or column can end up offset from each other.

You also need to design the layout of the individual items. You need this layout when you design the view holder, as described in the next section.

Implement your adapter and view holder

Once you determine your layout, you need to implement your Adapter and ViewHolder.

These two classes work together to define how your data is displayed. The ViewHolder is a wrapper around a View that contains the layout for an individual item in the list. The Adapter

creates ViewHolder objects as needed and also sets the data for those views. The process of associating views to their data is called binding.

When you define your adapter, you override three key methods:

onCreateViewHolder(): RecyclerView calls this method whenever it needs to create a new ViewHolder. The method creates and initializes the ViewHolder and its associated View, but does not fill in the view's contents—the ViewHolder has not yet been bound to specific data.

onBindViewHolder(): RecyclerView calls this method to associate a ViewHolder with data. The method fetches the appropriate data and uses the data to fill in the viewholder's layout. For example, if the RecyclerView displays a list of names, the method might find the appropriate name in the list and fill in the view holder's TextView widget.

getItemCount(): RecyclerView calls this method to get the size of the dataset. For example, in an address book app, this might be the total number of addresses. RecyclerView uses this to determine when there are no more items that can be displayed.

Code:

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:fitsSystemWindows="true">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</LinearLayout>
```

Student_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="10dp"
    android:orientation="horizontal">

    <ImageView
        android:id="@+id/imgStudent"
        android:layout_width="70dp"
        android:layout_height="70dp"
        android:src="@drawable/ic_launcher_foreground"/>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical"
        android:padding="10dp">

        <TextView
            android:id="@+id/txtName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Name"
            android:textSize="18sp"/>

        <TextView
            android:id="@+id/txtAddress"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
        android:text="Address"
        android:textSize="16sp"/>
```

```
</LinearLayout>
```

```
<Button
    android:id="@+id/btnDelete"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Delete"/>
```

```
</LinearLayout>
```

Student.java

```
package com.example.practical4;
```

```
public class student {
```

```
    String name;
```

```
    String address;
```

```
    int image;
```

```
    public student(String name, String address, int image) {
```

```
        this.name = name;
```

```
        this.address = address;
```

```
        this.image = image;
```

```
    }
```

```
}
```

studentAdapter.java

```
package com.example.practical4;
```

```
import android.view.LayoutInflater;
```

```
import android.view.View;
```

```
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

public class studentAdapter extends RecyclerView.Adapter<studentAdapter.ViewHolder> {

    ArrayList<student> students;

    public studentAdapter(ArrayList<student> students) {
        this.students = students;
    }

    @NonNull
    @Override
    public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {

        View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.student_item,
parent, false);

        return new ViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull ViewHolder holder, int position) {

        student student = students.get(position);

        holder.txtName.setText(student.name);
```

```
holder.txtAddress.setText(student.address);
holder.imgStudent.setImageResource(student.image);

holder.btnDelete.setOnClickListener(v -> {
    students.remove(position);
    notifyItemRemoved(position);
    notifyItemRangeChanged(position, students.size());
});
}

@Override
public int getItemCount() {
    return students.size();
}

public static class ViewHolder extends RecyclerView.ViewHolder {
    TextView txtName, txtAddress;
    ImageView imgStudent;
    Button btnDelete;

    public ViewHolder(@NonNull View itemView) {
        super(itemView);
        txtName = itemView.findViewById(R.id.txtName);
        txtAddress = itemView.findViewById(R.id.txtAddress);
        imgStudent = itemView.findViewById(R.id.imgStudent);
        btnDelete = itemView.findViewById(R.id.btnDelete);
    }
}
}
```

MainActivity.java

```
package com.example.practical4;

import android.os.Bundle;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {
    RecyclerView recyclerView;
    ArrayList<student> students;
    studentAdapter adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        recyclerView = findViewById(R.id.recyclerView);

        students = new ArrayList<>();

        students.add(new student("manav", "Surat", R.drawable.student));
        students.add(new student("alpesh", "Ahmedabad", R.drawable.student));
        students.add(new student("dharav", "Vadodara", R.drawable.student));
        students.add(new student("harsh", "Rajkot", R.drawable.student));

        adapter = new studentAdapter(students);

        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        recyclerView.setAdapter(adapter);
    }
}
```



Practical 5

AIM : Theme, Custom Styles, Drawables

Date:

Relevant CO: CO3

Theory :

The Android platform provides a large collection of styles and themes that you can use in your applications. You can find a reference of all available styles in the R.style class. To use the styles listed here, replace all underscores in the style name with a period. For example, you can apply the Theme_NoTitleBar theme with "@android:style/Theme.NoTitleBar".

Material Theming Properties

The Main 3 properties to achieve functional material theme in Android application are:

1. Color
2. Typography
3. Shape
4. Material Design Components

Primary Color Variants:

Attribute	Description	Default Value
colorPrimary	This color primary color of the application. This color is the most used color in the Application. This can also be called a branding color	#FF6200EE
colorPrimaryVariant	This is the variant of the colorPrimary. This may be a lighter or darker variant of the primary color.	#FF3700B3
colorOnPrimary	The name itself indicates the color should be assigned to those elements which are above the colorPrimary.	#FFFFFFFF

Applying Colors to Theme Attributes

Your color resource can then be applied to some theme attributes, such as the window background and the primary text color, by adding elements to your custom theme. These attributes are defined in your styles.xml file. For example, to apply the custom color to the window background, add the following two elements to your custom theme,

defined in MyAndroidApp/res/values/styles.xml

file :

```
<resources>

...

<style name="MyCustomTheme" ...>

<item name="android:windowBackground">@color/my_custom_color</item>

<item name="android:colorBackgroundCacheHint">@color/my_custom_color</item>

</style>

...

</resources>
```

Drawable:

When you need to display static images in your app, you can use the Drawable class and its subclasses to draw shapes and images. A Drawable is a general abstraction for something that can be drawn. The various subclasses help with specific image scenarios, and you can extend them to define your own drawable objects that behave in unique ways.

There are two ways to define and instantiate a Drawable besides using the class constructors:

- Inflate an image resource (a bitmap file) saved in your project.
- Inflate an XML resource that defines the drawable properties.

Create drawables from resource images

You can add graphics to your app by referencing an image file from your project resources. Supported file types are PNG (preferred), JPG (acceptable), and GIF (discouraged). App

icons, logos, and other graphics, such as those used in games, are well suited for this technique. To use an image resource, add your file to the `res/drawable/` directory of your project. Once in your project, you can reference the image resource from your code or your XML layout. Either way, it's referred to using a resource ID, which is the file name without the file type extension. For example, refer to `my_image.png` as `my_image`.

Code:**Activity_ main.xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:fitsSystemWindows="true">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="This is normal text"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is custom style text"
        style="@style/customText"/>

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```

    android:layout_marginTop="20dp"
    android:text="Normal Button"/>

```

```

<androidx.appcompat.widget.AppCompatButton
    android:id="@+id/customButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="5dp"
    android:text="Custom Button"
    android:textColor="@android:color/white"
    android:background="@drawable/button_bg"
    android:backgroundTint="@null"/>

```

```

</LinearLayout>

```

Style.xml (res → values → style.xml)

```

<resources>

    <style name="customText">
        <item name="android:textSize">20sp</item>
        <item name="android:textColor">#2196F3</item>
        <item name="android:textStyle">italic</item>
    </style>

</resources>

```

Themes.xml (res → values → themes.xml)

```

<?xml version="1.0" encoding="utf-8"?>

<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#FFC107"/>

```

```
<corners android:radius="25dp"/>
</shape>
```

JAVA:

```
package com.example.practical5;

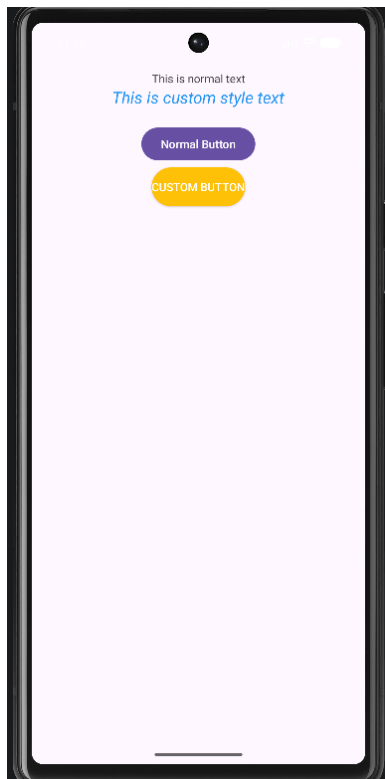
import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);
    }
}
```



Practical 6

AIM: Save user data in a database.

Date:

Relevant CO: CO3

Theory:

SQLite is another data storage available in Android where we can store data in the user's device and can use it any time when required. In this practical, we study how to create an SQLite database in an Android app and add data to that database.

Important Methods in SQLite Database

Below are the several important methods that we use in SQLite database integration in Android.

Method	Description
getColumnNames()	This method is used to get the array of column names of our SQLite table.
getCount()	This method returns the number of rows in the cursor.
isClosed()	This method returns a Boolean value when our cursor is closed.
getColumnCount()	This method returns the total number of columns present in our table.
getColumnName(int columnIndex)	This method returns the name of the column when we pass the index of our column.
getColumnIndex(String columnName)	This method returns the index of our column from the name of the column.
getPosition()	This method returns the current position of our cursor in our table.

Database - Creation

In order to create a database, call the `openOrCreateDatabase` method with the database name and mode as parameters. It returns an instance of `SQLiteDatabase`, which is received in an object. Its syntax is given below.

```
SQLiteDatabase mydatabase =  
openOrCreateDatabase("your database name", MODE_PRIVATE, null);
```

Database - Helper Class

For managing all the operations related to the database, an helper class called `SQLiteOpenHelper` is provided. It automatically manages the creation and update of the database. Its syntax is given below.

```
public class DBHelper extends SQLiteOpenHelper {  
    public DBHelper() {  
        super(context, DATABASE_NAME, null, 1);  
    }  
    public void onCreate(SQLiteDatabase db) {  
    }  
    public void onUpgrade(SQLiteDatabase database, int oldVersion, int newVersion) {  
    }  
}
```

Database - Insertion

We can create a table or insert data into a table using the `execSQL` method defined in the `SQLiteDatabase` class. Its syntax is given below.

```
mydatabase.execSQL("CREATE TABLE IF NOT EXISTS database(Uname VARCHAR,  
Password VARCHAR);");  
mydatabase.execSQL("INSERT INTO Tutorialspoint VALUES('admin', 'admin');");
```

Database - Fetching

We can retrieve data from the database using an object of the `Cursor` class. We call the `rawQuery` method of this class, which returns a result set with the cursor pointing to the table. We can move the cursor forward and retrieve the data.

```
Cursor resultSet = mydatabase.rawQuery("SELECT * FROM TutorialPoint", null);
resultSet.moveToFirst();
String username = resultSet.getString(0);
String password = resultSet.getString(1);
```

Code:**Activity_main.Xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginBottom="25dp">

        <ImageView
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:src="@drawable/images"
            android:contentDescription="College Logo"/>

        <TextView
            android:layout_width="0dp"
```

```
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:layout_marginStart="12dp"
    android:text="Shree Swami Atmanand Saraswati Institute of Technology"
    android:textSize="16sp"
    android:textStyle="bold"
    android:textColor="@color/design_default_color_primary_variant"
    android:gravity="center"/>
```

```
</LinearLayout>
```

```
<EditText
```

```
    android:id="@+id/studentName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Student Name"/>
```

```
<EditText
```

```
    android:id="@+id/studentEnroll"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Enrollment No"/>
```

```
<EditText
```

```
    android:id="@+id/studentSem"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Semester"/>
```

```
<Button
```

```
    android:id="@+id/addStudent"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
        android:text="Add Student"/>
    </LinearLayout>
```

MainActivity.java

```
package com.example.practical6;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText studentName, studentEnroll, studentSem;
    Button addStudent;
    DBHelper dbHelper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        studentName = findViewById(R.id.studentName);
        studentEnroll = findViewById(R.id.studentEnroll);
        studentSem = findViewById(R.id.studentSem);
        addStudent = findViewById(R.id.addStudent);

        dbHelper = new DBHelper(this);

        addStudent.setOnClickListener(v -> {
```

```
String name = studentName.getText().toString();
String enroll = studentEnroll.getText().toString();
String semester = studentSem.getText().toString();

dbHandler.addStudent(name, enroll, semester);

Toast.makeText(this, "Student Saved Successfully", Toast.LENGTH_SHORT).show();

studentName.setText("");
studentEnroll.setText("");
studentSem.setText("");
});
}
}
```

DBHandler.java

```
package com.example.practical6;

import android.content.ContentValues;
import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DBHandler extends SQLiteOpenHelper {

    private static final String DB_NAME = "CollegeDB";
    private static final int DB_VERSION = 1;

    public DBHandler(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
```

```
String query = "CREATE TABLE Student (" +
    "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
    "name TEXT, " +
    "enroll TEXT, " +
    "semester TEXT)";

db.execSQL(query);
}

public void addStudent(String name, String enroll, String semester) {

    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();

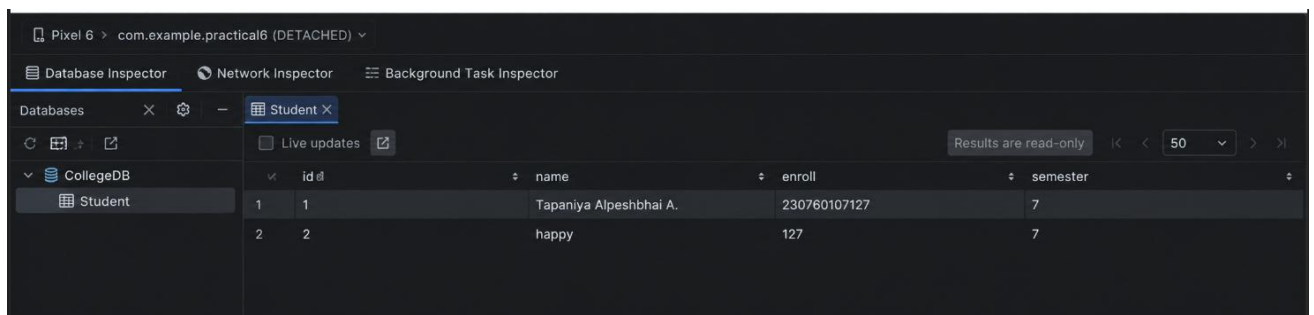
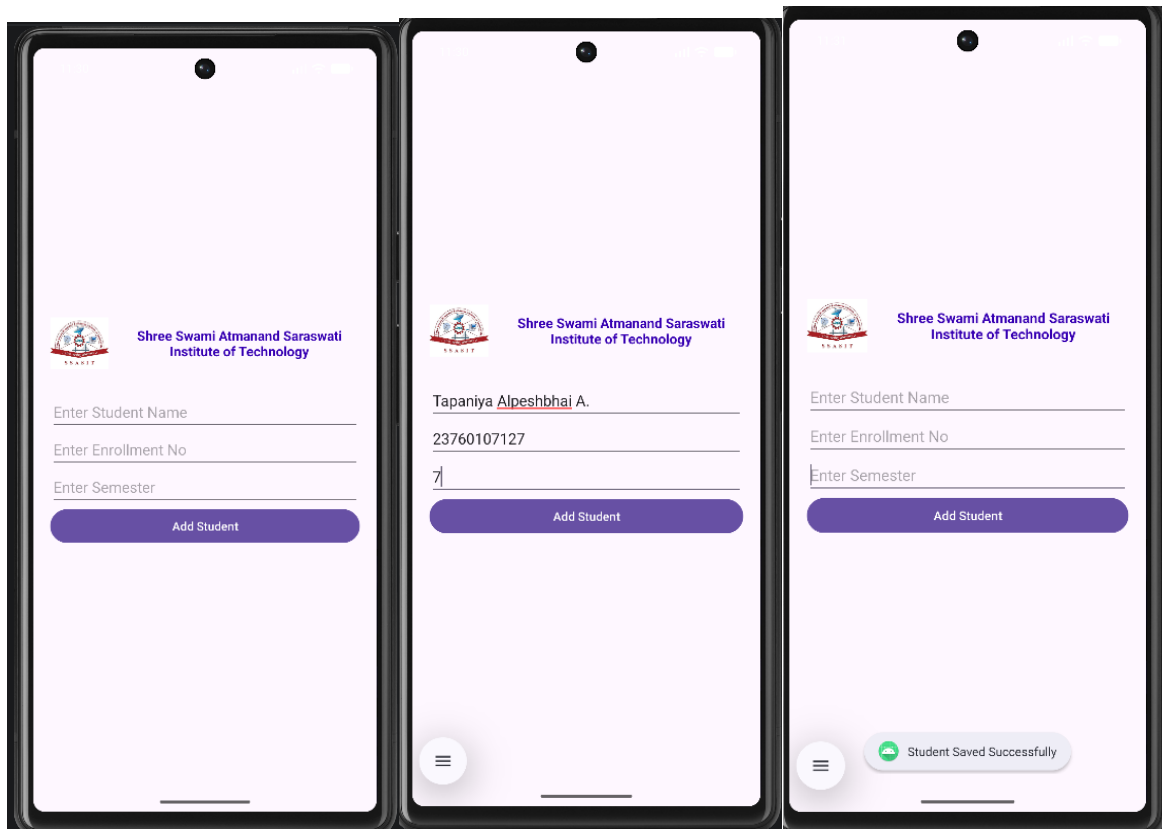
    values.put("name", name);
    values.put("enroll", enroll);
    values.put("semester", semester);

    db.insert("Student", null, values);

    db.close();
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

    db.execSQL("DROP TABLE IF EXISTS Student");
    onCreate(db);
}
}
```



Practical 7

AIM: Get and Save User Preferences.

Date:

Relevant CO: CO3, CO4

Theory:

One of the most interesting data storage options Android provides is Shared Preferences. Shared Preferences is used to store and retrieve small amounts of primitive data as key/value pairs in an XML file on the device storage. It can store values such as String, int, float and Boolean. Shared Preferences can be thought of as a dictionary or a key/value pair. For example, the key can be “username” and the value can be the user's username. The value can later be retrieved using the same key.

The SharedPreferences class provides APIs for reading, writing and managing this data. Shared Preferences are suitable for saving user settings or data that needs to be used in different activities within the app. For persistent data, preferences can be saved when the activity is paused and restored when the activity is created again. The data stored using Shared Preferences is kept private within the application.

When you want to get the values, call the `getSharedPreferences()` method. For security and backward compatibility, `MODE_PRIVATE` should be used.

`getSharedPreferences()`

This method takes two arguments: the name of the SharedPreferences file and the mode used to store the file.

```
public abstract SharedPreferences getSharedPreferences(String name, int mode)
```

Shared Preferences Modes

Mode	Description
MODE_PRIVATE	Keeps the SharedPreferences file private and secures the user's data. Recommended for use.

MODE_PUBLIC	Makes the file public and it may be accessible by other applications on the device.
MODE_APPEND	Used for appending data to an existing SharedPreferences file.

Methods of Shared Preferences

Method	Description
contains(String key)	Checks whether the preferences contain a particular key.
edit()	Creates an Editor to modify preference data.
getAll()	Retrieves all values from the preferences.
getBoolean(String key, boolean defValue)	Retrieves a Boolean value from the preferences.
getFloat(String key, float defValue)	Retrieves a float value from the preferences.
getInt(String key, int defValue)	Retrieves an integer value from the preferences.
getLong(String key, long defValue)	Retrieves a long value from the preferences.
getString(String key, String defValue)	Retrieves a String value from the preferences.
getStringSet(String key, Set<String> defValues)	Retrieves a set of String values from the preferences.
registerOnSharedPreferenceChangeListener(...)	Registers a callback that is invoked when a preference changes.
unregisterOnSharedPreferenceChangeListener(...)	Unregisters a previously registered preference-change callback.

Code:**Activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="USER REGISTRATION"
        android:textSize="24sp"
        android:textStyle="bold"/>

    <EditText
        android:id="@+id/username"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Username"/>

    <EditText
        android:id="@+id/password"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Password"
        android:inputType="textPassword"/>

    <Button
```

```
android:id="@+id/saveButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Save"/>
```

```
<Button
```

```
    android:id="@+id/loginButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"/>
```

```
</LinearLayout>
```

Activity_login.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
```

```
<TextView
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="USER LOGIN"
    android:textSize="24sp"
    android:textStyle="bold"/>
```

```
<EditText
```

```
    android:id="@+id/loginUsername"
    android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:hint="Enter Username"/>
```

```
<EditText
    android:id="@+id/loginPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Password"
    android:inputType="textPassword"/>
```

```
<Button
    android:id="@+id/getButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"/>
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.practical7;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText username, password;
    Button saveButton, loginButton;
    UserPreferences userPreferences;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    username = findViewById(R.id.username);
    password = findViewById(R.id.password);

    saveButton = findViewById(R.id.saveButton);
    loginButton = findViewById(R.id.loginButton);

    userPreferences = new UserPreferences(this);

    // Save Data
    saveButton.setOnClickListener(v -> {

        userPreferences.saveData(username.getText().toString(),
password.getText().toString()
        );

    });

    // Open Login Page
    loginButton.setOnClickListener(v -> {

        Intent intent = new Intent(MainActivity.this, LoginActivity.class
        );

        startActivity(intent);

    });
}
```

LoginActivity.java

```
package com.example.practical7;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class LoginActivity extends AppCompatActivity {

    EditText loginUsername, loginPassword;
    Button getButton;
    UserPreferences userPreferences;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_login);

        loginUsername = findViewById(R.id.loginUsername);
        loginPassword = findViewById(R.id.loginPassword);

        getButton = findViewById(R.id.getButton);

        userPreferences = new UserPreferences(this);

        getButton.setOnClickListener(v -> {

            String savedUsername = userPreferences.getUsername();
```

```
String savedPassword = userPreferences.getPassword();

String username = loginUsername.getText().toString();

String password = loginPassword.getText().toString();

if (username.equals(savedUsername) && password.equals(savedPassword)) {

    Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show();

} else {

    Toast.makeText(this, "Invalid Username or Password",
Toast.LENGTH_SHORT).show();
    }
});
}
}
```

UserPreferences.java

```
package com.example.practical7;

import android.content.Context;
import android.content.SharedPreferences;
import android.widget.Toast;

public class UserPreferences {

    SharedPreferences preferences;
    SharedPreferences.Editor editor;
    Context context;

    public UserPreferences(Context context) {
```

```
this.context = context;

preferences = context.getSharedPreferences("UserData", Context.MODE_PRIVATE);

editor = preferences.edit();
}

// Save Data here
public void saveData(String username, String password) {

    editor.putString("username", username);
    editor.putString("password", password);

    editor.apply();

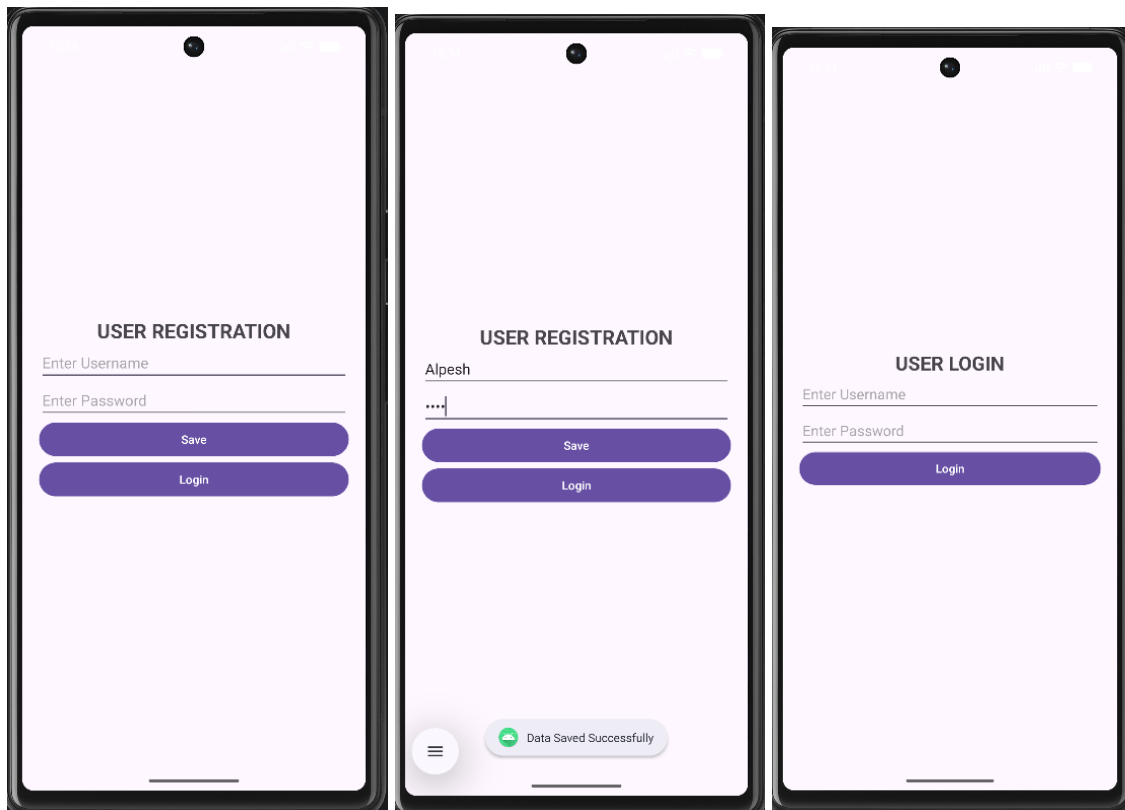
    Toast.makeText(context, "Data Saved Successfully", Toast.LENGTH_SHORT).show();
}

// Get Username
public String getUsername() {

    return preferences.getString("username", "");
}

// Get Password
public String getPassword() {

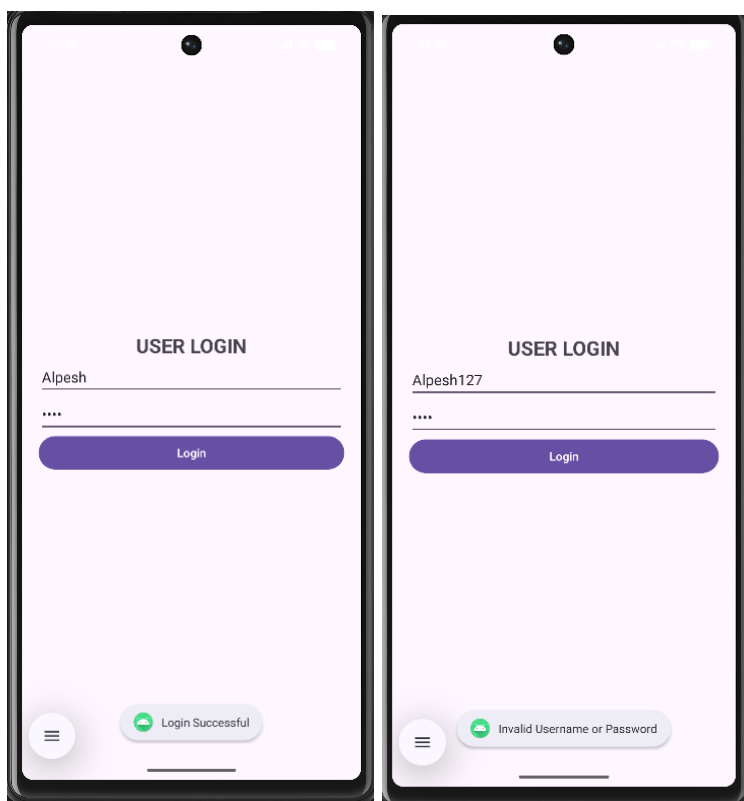
    return preferences.getString("password", "");
}
}
```



Registration page

Data Saved

Login Page



Login Successful

invalid check

Practical 8

AIM: make a use of android system.

Date:

Relavant CO: CO5

Theory :

You will use `MediaStore.ACTION_IMAGE_CAPTURE` to launch an existing camera application installed on your phone. Its syntax is given below

```
Intent intent = new Intent(android.provider.MediaStore.ACTION_IMAGE_CAPTURE);
```

Apart from the above, there are other available Intents provided by `MediaStore`. They are listed as follows

Intent type	Description
<code>ACTION_IMAGE_CAPTURE_SECURE</code>	It returns the image captured from the camera, when the device is secured
<code>ACTION_VIDEO_CAPTURE</code>	It calls the existing video application in android to capture video
<code>EXTRA_SCREEN_ORIENTATION</code>	It is used to set the orientation of the screen to vertical or landscape
<code>EXTRA_FULL_SCREEN</code>	It is used to control the user interface of the <code>ViewImage</code>
<code>INTENT_ACTION_VIDEO_CAMERA</code>	This intent is used to launch the camera in the video mode
<code>EXTRA_SIZE_LIMIT</code>	It is used to specify the size limit of video or image capture size

Code:**AndroidManifest.xml**

```
<!-- Camera Permission -->
<uses-permission android:name="android.permission.CAMERA" />

<!-- Camera Hardware -->
<uses-feature
    android:name="android.hardware.camera"
    android:required="false" />
```

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <Button
        android:id="@+id/cameraButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open Camera"/>

    <ImageView
        android:id="@+id/clickImage"
        android:layout_width="match_parent"
        android:layout_height="400dp"
        android:layout_margin="20dp"
        android:adjustViewBounds="true"
```

```
        android:scaleType="fitCenter" />
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.practical8;
```

```
import android.Manifest;
```

```
import android.content.Intent;
```

```
import android.content.pm.PackageManager;
```

```
import android.graphics.Bitmap;
```

```
import android.os.Bundle;
```

```
import android.provider.MediaStore;
```

```
import android.widget.Button;
```

```
import android.widget.ImageView;
```

```
import android.widget.Toast;
```

```
import androidx.activity.result.ActivityResultLauncher;
```

```
import androidx.activity.result.contract.ActivityResultContracts;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
import androidx.core.content.ContextCompat;
```

```
public class MainActivity extends AppCompatActivity {
```

```
    Button cameraButton;
```

```
    ImageView clickImage;
```

```
    // Camera permission
```

```
    ActivityResultLauncher<String> permissionLauncher =
```

```
        registerForActivityResult(
```

```
            new ActivityResultContracts.RequestPermission(),
```

```
            granted -> {
```

```
                if (granted) {
```

```
        openCamera();
    } else {
        Toast.makeText(
            this,
            "Camera Permission Denied",
            Toast.LENGTH_SHORT
        ).show();
    }
});

// Camera result
ActivityResultLauncher<Intent> cameraLauncher =
    registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {

            if (result.getResultCode() == RESULT_OK && result.getData() != null &&
result.getData().getExtras() != null) {

                Bitmap image = (Bitmap)
                    result.getData()
                        .getExtras()
                        .get("data");

                if (image != null) {
                    clickImage.setImageBitmap(image);
                }
            }
        });

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
```

```
setContentView(R.layout.activity_main);

cameraButton = findViewById(R.id.cameraButton);
clickImage = findViewById(R.id.clickImage);

cameraButton.setOnClickListener(v -> {

    if (ContextCompat.checkSelfPermission(
        this, Manifest.permission.CAMERA) ==
        PackageManager.PERMISSION_GRANTED) {

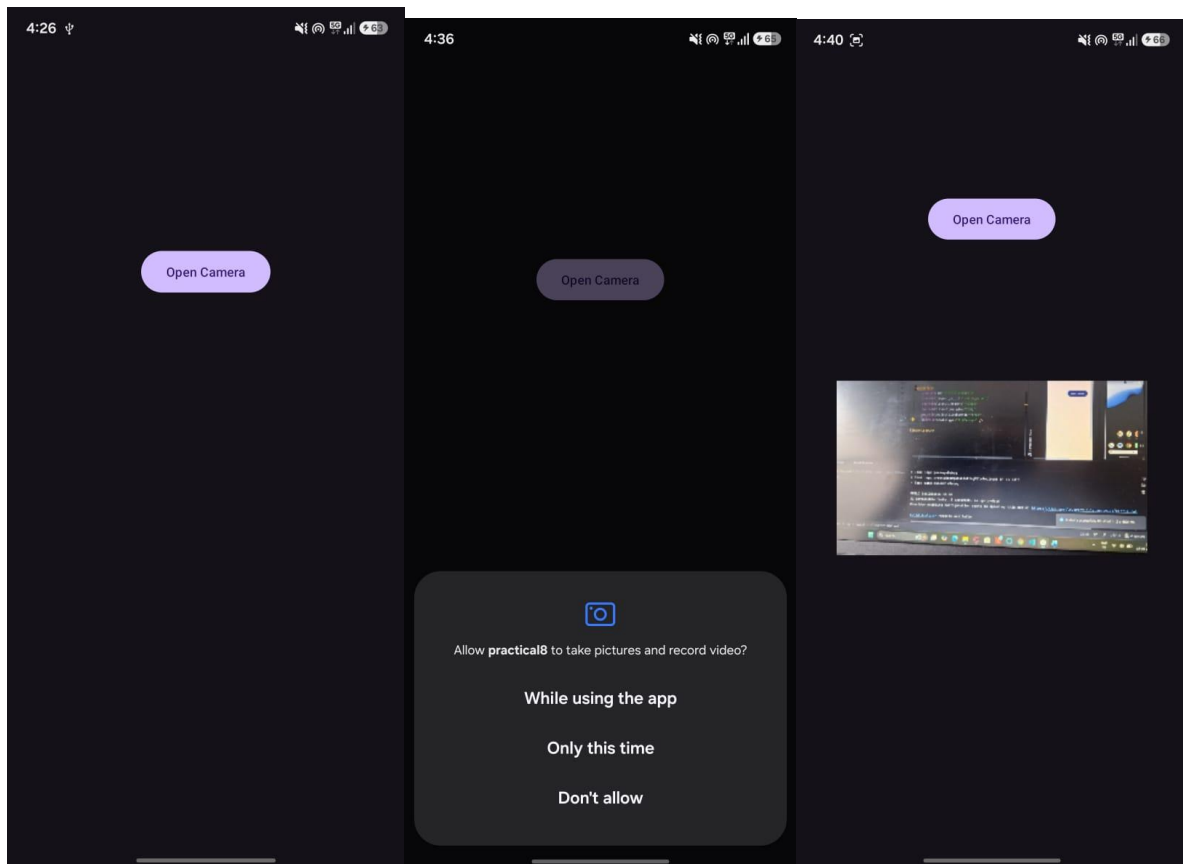
        openCamera();

    } else {
        permissionLauncher.launch(
            Manifest.permission.CAMERA
        );
    }
});
}

// Open Camera
private void openCamera() {

    Intent intent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);

    cameraLauncher.launch(intent);
}
}
```



Click Button

permission show

show output

Practical 9

AIM: Using location service get the current location and display in TextView.

Date:

Relevant CO: CO3, CO4

Theory :

A data class representing a geographic location. A location consists of a latitude, longitude, timestamp, accuracy, and other information such as bearing, altitude and velocity.

All locations generated through LocationManager are guaranteed to have a valid latitude, longitude, timestamp (both Unix epoch time and elapsed realtime since boot), and accuracy. All other parameters are optional.

Since using the user's permission is a matter concerned with high privacy, first acquire the user's permission to use their location by requesting them for it. From Android 6.0(Marshmallow), the concept of run-time permissions was rolled in and so the same will be used for getting permission. Any of the following permissions can be used for this:

ACCESS_COARSE_LOCATION: It provides location accuracy within a city block.

`<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>`

- In order to get the last location of the user, make use of the Java public class `FusedLocationProviderClient`. It is actually a location service that combines GPS location and network location to achieve a balance between battery consumption and accuracy. GPS location is used to provide accuracy and network location is used to get location when the user is indoors.
- In conjunction with `FusedLocationProviderClient`, `LocationRequest` public class is used to get the last known location. On this `LocationRequest` object, set a variety of methods such as set the priority of how accurate the location to be or in how many intervals, request of the location is to be made.
- If very high accuracy is required, use `PRIORITY_HIGH_ACCURACY` as an argument to the `setPriority(int)` method. For a city level accuracy(low accuracy), use `PRIORITY_LOW_POWER`.
- Once the `LocationRequest` object is ready, set it on the `FusedLocationProviderClient` object to get the final location.

Code:**AndroidManifest.xml**

```
<uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION"/>

<uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Current Location"
        android:textSize="24sp"
        android:textStyle="bold"/>

    <TextView
        android:id="@+id/latTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"
        android:text="Latitude"
        android:textSize="18sp"/>

    <TextView
        android:id="@+id/lonTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Longitude"
        android:textSize="18sp"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.practical9;

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;

public class MainActivity extends AppCompatActivity {

    TextView latTextView, lonTextView;

    FusedLocationProviderClient fusedLocationClient;

    int LOCATION_PERMISSION_CODE = 100;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        latTextView = findViewById(R.id.latTextView);
        lonTextView = findViewById(R.id.lonTextView);

        fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);

        getLocation();
    }

    private void getLocation() {

        if (ActivityCompat.checkSelfPermission(
            this, Manifest.permission.ACCESS_FINE_LOCATION) !=
```

```

PackageManager.PERMISSION_GRANTED) {

    ActivityCompat.requestPermissions(
        this,
        new String[]{
            Manifest.permission.ACCESS_FINE_LOCATION,
            Manifest.permission.ACCESS_COARSE_LOCATION
        }, LOCATION_PERMISSION_CODE);

    return;
}

fusedLocationClient.getLastLocation()
    .addOnSuccessListener(location -> {

        if (location != null) {

            latTextView.setText("Latitude: " + location.getLatitude());
            lonTextView.setText("Longitude: " + location.getLongitude());

        } else {

            Toast.makeText(this, "Location not available",
                Toast.LENGTH_SHORT).show();
        }
    });
}

@Override
public void onRequestPermissionsResult(
    int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {

    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

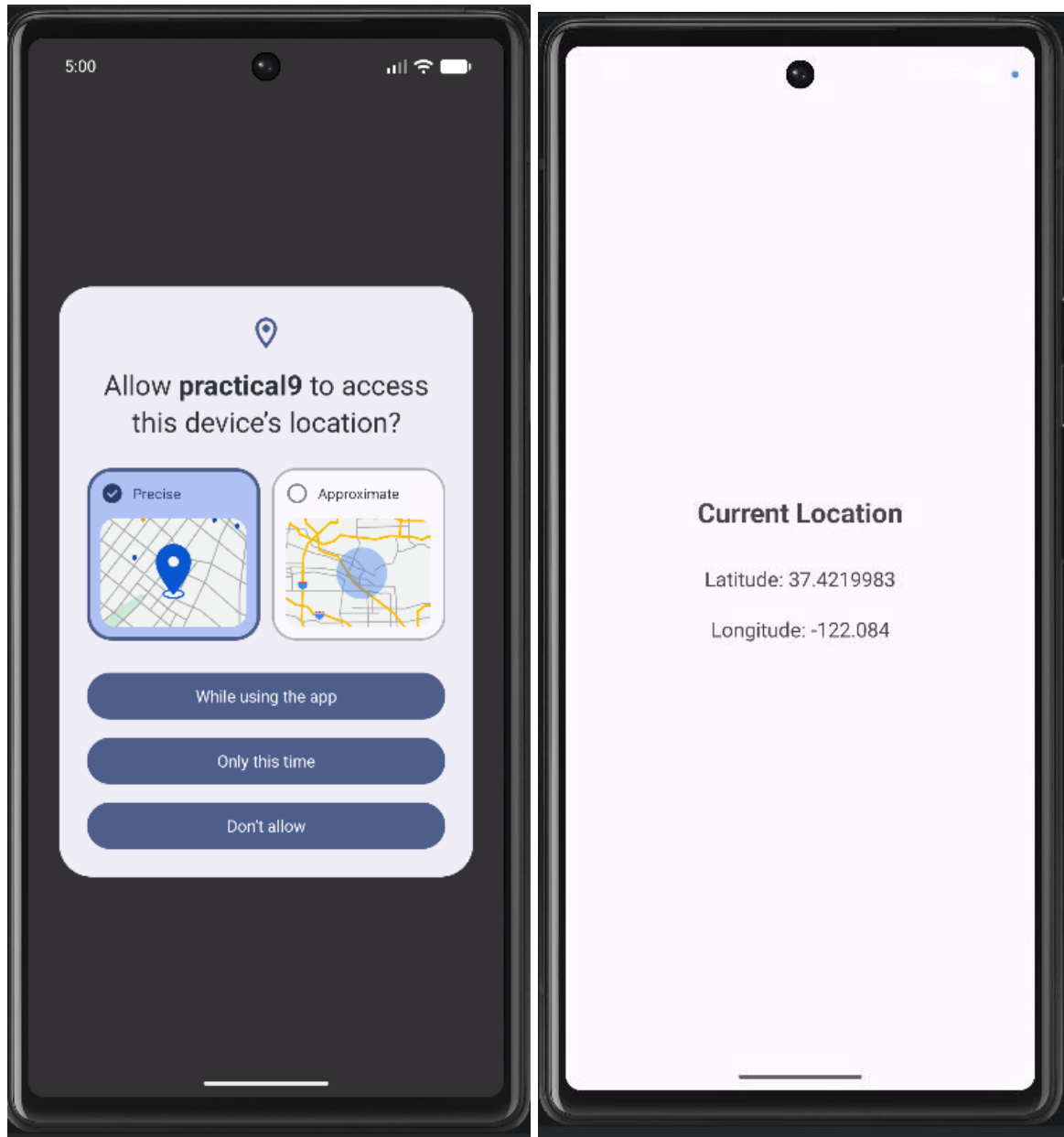
    if (requestCode == LOCATION_PERMISSION_CODE) {

        if (grantResults.length > 0 && grantResults[0] ==
            PackageManager.PERMISSION_GRANTED) {
            getLocation();

        } else {
            Toast.makeText(this, "Location Permission Denied",
                Toast.LENGTH_SHORT).show();
        }
    }
}

```

```
}  
}  
}  
}
```



Practical 10

AIM: Display the use of animations.

Date:

Relevant CO: CO4

Theory :

Animation is the process of adding a motion effect to any view, image, or text. With the help of an animation, you can add motion or can change the shape of a specific view. Animation in Android is generally used to give your UI a rich look and feel. The animations are basically of three types as follows:

1. Property Animation

Property Animation is one of the robust frameworks which allows animating almost everything. This is one of the powerful and flexible animations which was introduced in Android 3.0. Property animation can be used to add any animation in the CheckBox, RadioButtons, and widgets other than any view.

2. View Animation

View Animation can be used to add animation to a specific view to perform tweened animation on views. Tweened animation calculates animation information such as size, rotation, start point, and endpoint. These animations are slower and less flexible. An example of View animation can be used if we want to expand a specific layout in that place we can use View Animation. The example of View animation can be seen in Expandable RecyclerView.

3. Drawable Animation

Drawable Animation is used if you want to animate one image over another. The simple way to understand is to animate drawable is to load the series of drawable one after another to create an animation. A simple example of drawable animation can be seen in many apps Splash screen on apps logo animation.

Important Methods Of Animation:

Methods	Description
startAnimation()	This method will start the animation.
clearAnimation()	This method will clear the animation running on a specific view.

Code:**Activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:src="@drawable/images"
        android:contentDescription="@string/app_name" />

    <Button
        android:id="@+id/animatedButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Animate" />

</LinearLayout>
```

Blink.xml (res → anim (create resource directory) → blink.xml)

```
<?xml version="1.0" encoding="utf-8"?>

<alpha
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="1.0"
    android:toAlpha="0.0"
    android:duration="500"
    android:repeatMode="reverse"
    android:repeatCount="3" />
```

MainActivity.java

```
package com.example.practical10;
```

```

import android.os.Bundle;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.widget.Button;
import android.widget.ImageView;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Button animatedButton;
    ImageView imageView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        animatedButton = findViewById(R.id.animatedButton);
        imageView = findViewById(R.id.imageView);

        animatedButton.setOnClickListener(v -> {
            Animation animation = AnimationUtils.loadAnimation(this, R.anim.blink);
            imageView.startAnimation(animation);
        });
    }
}

```

